BRAIN BENCHMARK · TELEGRAM PLAYBOOK

Test the change.
Then decide what to ship.

A practical guide to dynamic Cost/History studies, brain parameters and code changes through **@phorvexbot**: write a precise request, inspect the tests, and compare your candidate with the baseline brain.

English edition | Copyable examples | Study, parameter and code workflows | HTML + CSV reports

Guide revision: **2026-10-31** · Dynamic Cost / History

Study walkthrough and JSON builder →

Quick start for file uploads

Send `/templates`, edit an example and attach it as a document. Put `/validate` in the file caption to check structure and values only. An empty caption submits valid experiments automatically. Read the full walkthrough →

01

What does the bot do?

Parameter sweeps, bounded code changes, and configured Cost/History comparisons.

The bot has three workflows: `/param` compares parameter values using the deployed brain; `/bench` generates a bounded code change and replays it on recorded states; a `study` file compares reference Cost/History algorithms with explicit arms and seeds.

Identify the experiment type first.

Replay evaluates decisions on past snapshots. A study compares registered reference algorithms on explicit recorded or synthetic inputs. Neither alone establishes live defense rates, realized cost or verified coverage for counterfactual decisions.

The bot does not apply these changes to the live brain. Retain the report, configuration and evidence type together. For Cost and History, start with the dynamic-study walkthrough.

02

Access and readiness

Set up access once, then run /check before an important experiment.

1

Open a private chat with the bot

Open @phorvexbot and press **Start**. If your account is not authorized yet, ask the bot owner for a fresh invitation, open it, and press Start. The owner can use `/invite` to create a one-use membership link valid for 24 hours.

Opening the public bot link does not grant membership. The bot does not reply to unauthorized accounts.

2

Get help, check dependencies, and download the playbook

Send /help and /check as separate messages. Use /guide whenever you want both English guide files attached in this chat:

First message · bot usage

```
/help
```

Second message · readiness check

```
/check
```

Download this English guide as HTML and PDF

```
/guide
```

3

Check the requirements for your workflow

Row in /check	Practical meaning
Telegram / allowlist / state	The bot, your authorization, and its job storage must be ready.
Dashboard / deployed brain-replay	The dashboard and verified replay identity are needed, along with ticks from the matching revision.
Operator repository	`/bench` needs the exact baseline brain commit in the server repository.
Free disk / Go / OpenAI	Needed to generate code, run tests and build a candidate. The default `/bench` free-disk threshold is 4 GiB.

A `WARN` can disable only one workflow. Read the row to see what is unavailable. An all-PASS readiness check does not guarantee that future tests or experiments will succeed.

Study prerequisites

An inline-trace study needs no LLM, Go build, source repository or dashboard. A recorded study requires a separately prepared dataset; /check does not check whether that dataset exists.

03 Choose your workflow

Three workflows with different inputs and evidence.

`study / JSON`

Compare Cost and History

Configure arms, seeds and registered algorithms in a file. No LLM code generation.

Walkthrough →

`/param`

Vary an existing parameter

Use the deployed replay binary’s registered overrides.

Walkthrough →

`/bench`

Make a bounded code change

Specify the file/function; generate a patch, run tests and replay it.

Walkthrough →

A History proposal, Cost Word document or long experiment description is not automatically an executable request. Use the format for your selected workflow.

Options for /param and /bench

Option	Meaning	Important detail
`policy=vllm/vllm-defense`	Choose a namespace and policy	Use `namespace/name`. This is the current default policy.
`ticks=64`	Request up to 64 samples for an initial check	Current default: 500. Allowed request: 1–5000. Actual coverage depends on available, eligible data.
`nosync`	Use the data and replay identity already on the server	Without this flag, the bot syncs first. After a new deployment, omit it to refresh the identity and data.

> **With nosync, “current” means the last identity cached on the server.**
>
> The cache may lag behind a recent live deployment. Check the baseline commit in the report. Sync does not guarantee that the matching source commit exists in the operator repository; missing source must be fetched separately.

04

Change parameters with /param

Explore a supported override without editing code.

1

Choose the parameter and hypothesis

Example: “If I change the allowed cost ceiling, how do selected moves and estimated disruption cost change?” `P-FEA-006`, the **CHi cost ceiling**, is relevant to that question.

The ceiling is in normalized disruption units, not dollars. A lower value makes the cost gate stricter; a higher value makes it more permissive. This relationship alone does not establish better defense.

2

Run a small comparison

Example experiment · up to 64 cached ticks

```
/param P-FEA-006 1.0 1.2 1.5 policy=vllm/vllm-defense ticks=64 nosync
```

The bot automatically adds a control arm using the **recorded effective value** and replays each value you supplied as another arm. You do not need to enter the control value. Do not assume that 1.2 is the current value without checking the report.

Use ASCII digits and a decimal point. Each command accepts 1–11 experiment values. All numbers in this guide are examples, not recommendations for configuring the live brain.

3 Read the report, then increase the sample

Check status, actual sample size, and each value against the control. If the data and result are interpretable, try a larger sample. This example also syncs fresh data:

Larger sample with a fresh sync

```
/param P-FEA-006 1.0 1.2 1.5 policy=vllm/vllm-defense ticks=500
```

The sample or baseline revision may change between runs. Read every experiment against its own control. Check the cohort and commit before comparing numbers from separate reports.

Known parameters and their limits

ID	Meaning and unit	Domain and limitations
`P-FEA-006`	CHi ceiling; normalized disruption	Greater than 0, up to 1000. No `unset` override.
`P-FEA-005`	Proactive hygiene cadence; seconds	Whole seconds from 1 to 604800. The override changes both cadence and exposure-pressure scale. Fresh trusted event evidence can bypass the cadence gate.
`P-OBJ-003`	Fixed disruption aversion, or beta	0–10 and `unset`. Incompatible with `costPreferences`; preflight can return `MODE_CONFLICT`.

Second example · 5, 15 and 30 minutes, expressed in seconds

```
/param P-FEA-005 300 900 1800 policy=vllm/vllm-defense ticks=64 nosync
```

An ID in the registry does not mean every parameter can be overridden. The replay binary must support it and preflight must accept it. This cadence example is not a uniform throttle on every attack response.

05 Change code and formulas with /bench

Turn a clear hypothesis into a patch, tests, and a three-arm comparison.

> **Use study JSON for the registered Cost/History experiments.**
>
> This chapter is about changing brain code. Its 20% norm-multiplier example does not implement the endpoint-second model or History v1.1. Study walkthrough

/bench takes a description of a change.

In the current interface, you describe what to change and a model generates a diff. There is no direct branch or commit selector, uploaded-diff execution, or `/rerun` command.

1 Define one question you can evaluate

A useful example: “If I increase the norm term's weight in the cost penalty by 20%, does estimated disruption cost decrease? What happens to selected moves, predicted coverage, and no-op under attack?”

“Make the brain better” leaves the scope and evaluation unclear. Limit your first experiment to one function or condition. If you do not know the file or function, locate it in the source for the baseline revision first.

2 Include these five parts in the request

Part	Cost-penalty example
Location	`phorvex/internal/model/economics.go`, function `EconomicsMetadata.CostPenalty`
Mode	Only `independent-cost-preferences-v1`
New formula or behavior	Change the norm multiplier from 1 to 1.2; keep the flow term as it is.
Constraints	Preserve legacy and fixed-beta behavior, feasibility gates, SLO checks, and budgets.
Change size	Keep the patch minimal, without unrelated edits or broad refactoring.

Use exact file paths, function names and code identifiers. The technical English examples help retrieve relevant context. A fully Persian request without code identifiers may retrieve too little useful context. Verify the example's path and formula against your baseline commit.

3 State the old and new formulas explicitly

This is an example of defining an experiment. The suitability of 1.2 and whether the change passes existing tests have not been established.

Before · only in independent-cost-preferences-v1 mode

```
penalty = w_norm * norm + w_flow * flow
```

After · increase the norm contribution to the penalty by 20%

```
penalty = 1.2 * w_norm * norm + w_flow * flow
```

For arithmetic illustration only: with `w_norm=0.5`, `norm=2`, `w_flow=0.3` and `flow=1`, the penalty increases from 1.3 to 1.5. This describes the expression itself. The brain's move count or final estimated cost will not necessarily change by 20%.

4 Send the complete command as one message

Formula-change example · copyable experiment

```
/bench policy=vllm/vllm-defense ticks=64 nosync In phorvex/internal/model/economics.go, change EconomicsMetadata.CostPenalty in independent-cost-preferences-v1 mode from m.NormPreference.Value*norm + m.FlowPreference.Value*flow to 1.2*m.NormPreference.Value*norm + m.FlowPreference.Value*flow. Preserve legacy and fixed-beta behavior, feasibility gates, SLO checks and budgets. Keep the patch minimal.
```

> **Put options before the change description.**
>
> The order is `/bench`, then `policy=...`, `ticks=...` and optionally `nosync`, followed by the description. Options at the end can be interpreted as part of the model request instead.

To refresh data and the replay identity before the experiment, omit `nosync`. Use the command builder to change the sample size or other options.

5 Follow the execution stages

The bot gives you a job ID and reports progress. One job runs at a time; up to five can wait in the queue. Cold builds and tests can take several minutes. There is no fixed completion time.

Check the current job and queue

```
/status
```

Stage	What the bot does
Baseline and data selection	Checks replay identity and SHA256, selects ticks from the matching commit, and freezes the selected data for this experiment.
Patch generation	The model proposes a change against the baseline commit. The candidate is created in a temporary clone.
Tests	Runs Go tests on both versions: `internal/decision`, `cmd/brain-replay`, and changed packages. This is not the full repository CI suite.
Build	Builds baseline and candidate replays. Both use the same compiler release, read from the deployed binary's build metadata.
Replay and reporting	Replays three arms on the same ticks, then sends HTML, CSV, the diff and test results.

A failed test stops the job before the numerical benchmark.

If baseline or candidate tests fail, the bot sends the corresponding log. An intentional formula change can conflict with an existing numerical contract. Do not remove or bypass tests merely to get a benchmark result.

6 Keep all four outputs together

File	What to inspect
`report-<job>.html`	Status, commits, data coverage, fidelity, and comparison metrics
`metrics-<job>.csv`	Values, populations, intervals, data availability, and comparison type
`candidate-<job>.diff`	Whether the generated code implements exactly what you requested
`tests-<job>.json`	Baseline and candidate tests, packages, result counts, and toolchain release

File names include the job ID. Read the diff before interpreting the numbers. A successful build does not establish that the model understood your request correctly.

Every /bench request starts independently.

A second command does not continue the first candidate. It creates a fresh candidate from the baseline commit. Resending the same description with `ticks=500` does not guarantee the same patch. Compare the diffs. Replaying a fixed candidate deterministically requires a separate engineering workflow.

First run: check the pipeline without changing behavior

To exercise patch generation, tests, builds and report delivery, request a comment-only change. This is a pipeline check, not an optimization. Conceptually, zero changed decisions is the expected result.

Pipeline check · add a comment only

```
/bench policy=vllm/vllm-defense ticks=64 nosync In phorvex/internal/decision/ablation.go, add one explanatory comment immediately before the existing if base.Ratio > 0 branch, explaining that the positive-ratio check avoids division by zero. Change comments only. Preserve all executable code and behavior.
```

Allowed candidate scope: Go files under `phorvex/`, excluding `phorvex/tools/` and `_test.go` files. Dependency, `go.mod`, chart and Dockerfile changes, and file deletions are not accepted by this workflow.

06

Run experiments from a file

Upload Markdown, TXT or Word with bench, params, or both sections.

1

Download a completed example

This command sends eight files: five code/parameter examples and three Cost/History JSON studies. This chapter covers the code/parameter formats; study JSON is separate.

Download completed file examples

```
/templates
```

Use .md, .txt or .docx. Export Google Docs as Word (.docx) or plain text. Legacy binary .doc is not supported. Maximum document size: 2 MiB; extracted prose: 256 KiB. Study JSON may use the full 2 MiB.

2

Fill the sections you want to run

A file with both sections produces independent experiments in file order. The comment-only bench example below checks the pipeline; the parameter experiment compares trial ceilings. All values are illustrative.

Case A · both-sections.md or the equivalent Word paragraphs

```
# Independent brain experiments

## Bench
policy: vllm/vllm-defense
ticks: 64
sync: false
change:
In phorvex/internal/decision/ablation.go, add one explanatory comment immediately
before the existing if base.Ratio > 0 branch, explaining that the positive-ratio
check avoids division by zero. Change comments only. Preserve executable code.

## Params
policy: vllm/vllm-defense
ticks: 64
sync: false
P-FEA-006: 1.0, 1.2, 1.5
```

Sections are optional; present sections must be valid.

Omit bench to run only parameter experiments, or omit params to run only the code experiment. An empty or malformed section rejects the entire file. Parameters run as separate comparisons with their own controls; they are not applied to the code candidate.

Case B · params-only.md; each parameter ID creates one experiment

```
# Parameter comparisons

## Params
policy: vllm/vllm-defense
ticks: 64
sync: false
P-FEA-006: 1.0 1.2 1.5
P-FEA-005: 300 900 1800
```

Case C · bench-only.txt; a proposed formula change may fail existing tests

```
[bench]
policy: vllm/vllm-defense
ticks: 64
sync: false
change:
In phorvex/internal/model/economics.go, change EconomicsMetadata.CostPenalty in
independent-cost-preferences-v1 mode from
m.NormPreference.Value*norm + m.FlowPreference.Value*flow to
1.2*m.NormPreference.Value*norm + m.FlowPreference.Value*flow.
Preserve legacy and fixed-beta behavior, feasibility gates, SLO checks and budgets.
Keep the patch minimal.
```

Put section options before `change:`. Its description can continue over several lines. `sync: false` uses cached data; `sync: true` syncs first. An omitted option uses the bot default. Parameter trial values may be separated by commas or spaces.

3 Validate first, or upload to execute

Save the example text in a file and attach it as a **document** in the bot chat, rather than pasting the whole file as a text message. To inspect it without running anything, put `/validate` in the file's **caption**. To run it automatically after validation, send the document with an empty caption.

What you send	What happens
Valid file, empty caption	The whole file is parsed and validated, then queued as one batch.
Valid file, /validate caption	Structure and parameter domains are checked. No experiment is queued.
File with one invalid section/value	Rejected with an error and line number where available. Nothing is queued.
File containing only bench or only params	Only the supplied valid section is executed.
Malformed Word, oversized file, unsupported type	Rejected; use the supplied examples or export a fresh .docx/.txt.

Put /validate on the attachment itself.

Sending /validate as a separate message only displays instructions. It does not change the mode of the next upload. Keep section names, option keys, IDs and numeric digits exactly as shown in the examples.

Structural validation checks format, options, parameter IDs and registry value domains. It cannot establish that a proposed code change compiles or passes tests. Each experiment still runs its Go tests and/or runtime replay preflight.

4

Follow the batch and collect its results

One batch can contain up to five experiments. They execute sequentially on the host. Each experiment returns its usual report files; the batch also returns JSON and CSV summaries with job IDs and statuses. A runtime failure is reported and other independent experiments continue. Use `/cancel` to cancel the running batch and skip its remaining experiments.

Progress stays visible while work runs.

A single editable activity card shows the current task, elapsed time and batch experiment number. It refreshes every eight seconds while waiting for a model, tests, builds or replay. The moving bar is an activity indicator, without an invented percentage or ETA. Terminal states include DONE, PARTIAL, NEEDS_DATA, FAILED, CANCELLED and INTERRUPTED.

Illustrative activity card; not a live job

```
⁖ BATCH job ab12cd34ef56 — RUNNING
[□□■■■□□□□□] Experiment 2/3 (bench): running Go unit tests: candidate
Elapsed: 3m 10s | Current task: 42s
Activity indicator; no completion-time estimate.
```

PARTIAL in a batch summary means some experiments succeeded and others failed. PARTIAL in an individual experiment report refers to incomplete data coverage. Read these statuses separately.

Alternative: a command-list file

Use one complete command per line. For multiline descriptions, use the section format above.

Case D · command-list.txt

```
/param P-FEA-006 1.0 1.2 1.5 policy=vllm/vllm-defense ticks=64 nosync
/bench policy=vllm/vllm-defense ticks=64 nosync In phorvex/internal/decision/ablation.go, add one explanatory comment immediately
before the existing if base.Ratio > 0 branch, explaining that the positive-ratio check avoids division by zero. Change comments only.
Preserve all executable code and behavior.
```

Example rejection: `P-FEA-006: 0` is outside the allowed domain; `ticks: 5001` is invalid; `[bench]` without a change is incomplete. Fix every reported issue before resubmitting.

A study file is one experiment containing comparison arms. Legacy sections remain independent experiments; a History section does not stack on a previous Cost candidate. Study PARTIAL can indicate missing inputs or selector fallback.

Dynamic Cost and History benchmarks

07

One JSON file: configured comparison arms, recorded seeds and shared reports, without code generation.

This workflow is deployed.

A study executes registered reference algorithms using the supplied configuration. Change arms, selector settings and seeds without asking a model for another patch. A new, unregistered algorithm still requires implementation.

1. Choose one of the three ready-made files

Send `/templates`. Start with a synthetic example to learn the workflow. `/study` displays help; attaching the JSON file starts an experiment.

File	Comparison	Evidence
`study-cost-synthetic.json`	Legacy versus endpoint-v1 cost, the same deterministic B=1 selector, three seeds	Synthetic; 6 arm/seed runs
`study-history-synthetic.json`	Deterministic, entropy, score-only stochastic and history-weighted selectors; three seeds	Synthetic; 12 arm/seed runs
`study-history-recorded.json`	The same four selectors on a prepared recorded dataset	Recorded; requires a valid trace and ledgers

A synthetic example is not a production outcome.

The two synthetic files demonstrate computation and history behavior. The recorded request returns `NEEDS_DATA` until the required evidence is installed. Existing legacy snapshots or an all-PASS `/check` do not establish that these ledgers are available.

2. Configure and validate the request

Download a file from the builder below, or edit its JSON. Preserve the keys and identifiers. Attach it with `/validate` as the caption; after PASS, attach the same file with an empty caption to run.

Setting	Meaning
`arms`	1–8 named arms. The first arm is the decision-comparison reference; all arms share the input trace.
`seeds`	1–16 distinct integers. With identical input, configuration and implementation, draws are reproducible.
`epsilon`	Maximum expected score loss, not a guarantee for each draw. Zero permits only maximal-score choices.
`history.window_seconds`	Counter lookback window; it does not alter the mandatory 24-hour coverage contract.
`history.mode`	`observed` consumes recorded events. `simulated-starts` uses explicit synthetic acknowledgements and requires synthetic evidence.
`trace` **or** `dataset`	Supply exactly one: inline trace data, or `"dataset":"recorded"` for a prepared server dataset for this policy.

Limits: 2 MiB per JSON file, up to 5,000 ticks and 100,000 candidate rows across all arms and seeds. Do not wrap JSON in `[bench]`. Legacy `ticks` and `sync` are not top-level study fields.

3. Choose the cost model and selector

Choice	Meaning
`cost.kind: provided`	Consume authoritative c and rho inputs; useful for isolating selectors with a fixed cost estimator.
`cost.kind: legacy`	Combine supplied norm/flow terms with explicit weights and a chi gate. Values are dimensionless.
`cost.kind: endpoint-v1`	Duration estimates and endpoint-second reservations, including cold/censored data and quarantine-specific accounting.
`selector.kind`	`deterministic`, `entropy` or `history`. The History example includes a score-only arm with history weights set to zero.

For a History comparison, hold cost and gates fixed across selectors. For a Cost comparison, hold selector and move count fixed. This workflow uses B=1. Example parameters are study choices, not approved production defaults.

4. Supply meaningful history and recorded evidence

- `F` counts actual optional opportunities after gates; a retry is not another opportunity.
- `S` counts acknowledged execution starts. A draw, failed dispatch or proposed choice does not increment S.
- Security effects require matching verified events. An apparently successful action does not automatically earn verified coverage.
- Each arm/seed has isolated history. In observed mode, counterfactual choices do not generate actual execution events.

- Missing required history triggers deterministic `PICKER_UNAVAILABLE` fallback and `PARTIAL`. Missing cost accounting is not initialized to unused budget.

For recorded mode, the data owner must prepare and import chronological identities, gate verdicts, budgets, duration samples and valid history events. Renaming synthetic data does not make it recorded evidence. This reference runner consumes the required gate/scheduler verdicts; it does not rerun production schedulers or actuators.

Complete recorded-data request

This is a structurally complete file. Without the prepared dataset, its correct outcome is `NEEDS_DATA`.

study-history-recorded.json

```
{
  "schema": "phorvex-bench.study.v1",
  "title": "History v1.1 on normalized recorded evidence",
  "policy": "vllm/vllm-defense",
  "dataset": "recorded",
  "seeds": [7, 19, 41],
  "history": {"mode": "observed", "window_seconds": 3600},
  "arms": [
    {"id": "deterministic", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "deterministic"}},
    {"id": "entropy", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "entropy", "tau": 1, "epsilon": 0.5}},
    {"id": "score-only", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "history", "epsilon": 0.5, "score_scale": 1,
"eta_pair": 0, "eta_action": 0, "eta_fact": 0, "kappa_pair": 1, "kappa_action": 1, "q_ref": 1, "x0": 0.7, "a0": 0}},
    {"id": "history", "cost": {"kind": "endpoint-v1"}, "selector": {"kind": "history", "epsilon": 0.5, "score_scale": 1, "eta_pair":
2, "eta_action": 1, "eta_fact": 0, "kappa_pair": 1, "kappa_action": 1, "q_ref": 1, "x0": 0.7, "a0": 0}}
  ]
}
```

08 Build a copyable command

Choose your options, then send the output in Telegram.

This tool generates command text locally in this page. It does not connect to the bot or server. The example values are for learning, not production configuration recommendations.

The form checks ordering and some input constraints. File availability at the baseline commit, model output correctness, binary support, and preflight success are checked by the bot itself.

09 How to read the report

Establish comparison validity, examine metric changes, then assess your hypothesis.

Study reports: read evidence and status first

Status	Meaning
`COMPLETE`	The reference experiment completed with adequate inputs; this is not algorithm approval or a measured live outcome.
`PARTIAL`	Some ticks lack inputs or a selector fell back to deterministic behavior. Inspect gap and fallback counts.
`NEEDS_DATA`	The dataset or required evidence is unavailable. Supply the missing data.
`CANCELLED`	Cancellation was applied; partial evidence may be retained.

File / metric	Use
`study-<job>-report.html`	Arm/seed summary, evidence type, data gaps and paired decisions.
`study-<job>-metrics.csv`	Usable ticks, fallback counts, selections and expected score loss.
`study-<job>-result.json`	Machine-readable status, evidence gaps and available/unavailable metrics.
`study-<job>-evidence.zip`	Frozen input/manifest, candidates.csv, ticks.csv and per-candidate probabilities.

> More changed decisions do not mean improvement. Do not calculate a cost reduction between dimensionless legacy values and endpoint-seconds. Null/unavailable is not zero. Counterfactual verified_coverage and realized_cost are not observed; choices or summed reservations do not establish actual coverage or spend.

Below: replay reports for /bench and /param

1 Check status and actual data coverage

Report field	Meaning and next step
`COMPLETE`	The comparison finished with the expected coverage. This label alone does not establish improvement.
`PARTIAL`	Some data was ineligible or could not be compared. Read the denominators and exclusion reasons; do not treat this as equivalent to full coverage.
`FAILED / INVALID`	The result is not valid evidence for accepting the candidate. Resolve the reported problem first.
`64/64` **or similar**	Read actual counts separately for control and each arm. Requesting 64 or 500 ticks does not guarantee that many valid samples.

2 Understand the three /bench arms

`control`

The deployed replay binary

A tick is control-eligible only if this binary exactly reproduces the recorded production output.

`base-local`

The baseline source, built here

For a valid comparison, fidelity must be `EXACT` and its changed-decision share against control must be zero.

`candidate`

The code generated for your request

This is a different brain revision. Its results are labeled `OBSERVATIONAL`.

> **If base-local is DIVERGED, do not use the candidate result to make a decision.**
>
> First establish why the local baseline differs from the deployed binary. EXACT describes decision reproduction on the inspected ticks; it does not by itself guarantee that both binary files are byte-identical.

`/param` normally reports `MATCHED_REPLAY`. `OBSERVATIONAL` is expected for `/bench` because it compares different source revisions.

3 Read the metrics against your experiment question

ID	What it measures	How to interpret it
`M-P21`	Share of ticks whose selected action set differs from control	Behavioral sensitivity, with no inherent good/bad direction. It is not an improvement score.
`M-P26`	Selected moves per tick	A reduction may remove unnecessary moves or omit a needed response.
`M-P28`	Estimated disruption cost of selected moves	A cost-model estimate, not measured live disruption or money spent.

ID	What it measures	How to interpret it
`M-P19`	Share of attack-backed asset-ticks with no selected move	An increase calls for inspecting no-op reasons and potentially missed responses.
`M-P22`	Share of attack-backed opportunities with complete evidence and an admitted positive-gain action	Describes a modeled opportunity, not a successful live defense.
`M-P25`	Predicted coverage using common control-arm weights	Useful for comparing modeled coverage; it is not an observed security outcome.

Metrics do not all have the same population or denominator. `n/a` means unavailable, not zero. Read intervals and sample counts alongside the headline values.

4 Separate changed behavior from improvement

A wholly hypothetical example for learning how to interpret results. These numbers do not come from a real bot run.

Metric	Control	Candidate	What this metric alone says
Changed decisions `M-P21`	0%	25%	One in four compared ticks has a different selected set.
Moves per tick `M-P26`	0.8	0.6	Mean selected moves are 25% lower.
Estimated cost `M-P28`	0.40	0.32	Mean modeled cost is 20% lower.
No-op under attack `M-P19`	10%	15%	Up 5 percentage points; inspect possible adverse effects.

> **This does not establish that "the brain improved by 25%."**
>
> Estimated cost decreased, but no-op under attack also increased. Interpret the metrics together with the hypothesis and safety constraints. A single number cannot determine release acceptance.

In the CSV, `ARM_LEVEL` is the arm's value and `PAIRED_DIFF_VS_CONTROL` is a difference over matched pairs. Keep them distinct. `M-P18` uses each arm's own objective pricing, so comparing raw values after changing the cost formula can be misleading.

A recorded pipeline verification, with no optimization claim

In the recorded check on **2026-10-28**, job `0deacfd4c571` completed on baseline `0058a3edd2eb` with a comment-only candidate: 64/64 ticks, EXACT baseline fidelity, and 398 PASS test events, 2 SKIPs and zero failures per version. Changed decisions were zero. This is historical evidence that the pipeline ran. Use /check and a fresh report to assess the current state.

10

Troubleshooting

Keep the job ID, failed stage and error message together.

Symptom or message	Likely meaning	Next step
`NEEDS_DATA`	Required study evidence is missing.	Read result.json and data_gaps; prepare the recorded dataset. Repeating the same request does not create evidence.
`PICKER_UNAVAILABLE`	Required history or fact inputs are incomplete.	Check lookback coverage and events; do not present fallback choices as a full History result.
`Study validation / unknown fields`	An input key, type, seed or limit is invalid.	Start from a JSON template. Duplicate seeds, NaN and top-level ticks/sync are not accepted.
`no in-scope diff … model declined`	The /bench planner returned no permissible patch; the benchmark has not run.	Read plan-<job>.json. Use study JSON for supported Cost/History comparisons; use a bounded, precise request for code changes.
File rejected / section or line error	The format, key, value or an empty section is invalid.	Start with /templates, fix every issue, and attach the file with /validate to check again. Do not translate the example keys.
File validation PASS followed by a failed experiment	The structure was valid, but runtime preflight, tests or build failed.	Read that job's report and log. Structural validation does not guarantee passing code or adequate data.
PARTIAL in a batch summary	Some experiments succeeded and others failed.	Read batch JSON/CSV for each job ID and status. This differs from PARTIAL data coverage in an individual report.
The bot does not reply	Your account is unauthorized, the invitation is used/expired, or the service is unavailable.	Try the private chat and ask the owner for a fresh invitation. If you are already authorized, the service needs inspection.
`BOT_REPO does not contain the deployed commit`	The new binary exists, but its exact source commit is missing from the server clone.	The operator should fetch that commit into BOT_REPO, then rerun `/check` and start a new experiment. A tag is not required.

Symptom or message	Likely meaning	Next step
`no recorded ticks` **or revision mismatch**	No matching-revision data is available for the chosen policy.	Check the policy and run without `nosync`. If recording is inactive, inspect that policy's capture setup.
Free disk below the threshold	The `/bench` start guard refused the job.	Free space on the server, then run `/check`. Lowering the guard alone does not solve a space shortage.
`MODE_CONFLICT`	The parameter conflicts with the current economics mode, such as beta with costPreferences.	Choose a compatible parameter. Do not change the live economics mode just to force this experiment.
Invalid value / unsupported parameter	The ID, domain, unit, or override support is wrong.	Use ASCII numbers, the right unit and a supported parameter. Read the preflight message.
`LLM step failed` **or refused patch**	There is a model-response, context, or file-scope problem.	Supply an exact file/function and a smaller change. Remove out-of-scope edits. Authentication/quota errors need service-configuration inspection.
`base-local Go tests FAILED`	Even the baseline failed tests on the server.	Inspect the baseline log. Code, tooling or resource pressure may be responsible. Do not attribute this directly to the candidate formula.
`candidate Go tests FAILED`	The candidate violated an existing test.	Read the log. The change may be wrong, or the intended contract may need independent code and test review. Do not bypass tests to obtain numbers.
`build failed`	Compilation, toolchain, module or resource problem	Inspect the failed build stage and log. There is no valid benchmark result yet.
`DIVERGED`	The rebuilt baseline does not reproduce control decisions.	Stop interpreting candidate comparisons and resolve baseline fidelity first.
`n/a` **or incomplete coverage**	Required evidence, a valid denominator or replayable samples are missing.	Read data_status, reason and sample counts. Do not record `n/a` as zero.

For troubleshooting, keep these items:

Job ID, the submitted command, the failed stage, and that job's log or report. Resolve the first failure before queuing several similar expensive experiments.

11

Review checklist

This supports a review. Completing it does not automatically authorize a release.

- [ ] I selected study, param or bench and defined the question and metric units.

- [ ] I checked synthetic versus recorded evidence, source identity and input completeness.
- [ ] For a study, I retained the arms, seeds, shared input and comparison reference.
- [ ] For History, I distinguished opportunities, acknowledged starts and verification, and inspected fallback.
- [ ] For /bench, I reviewed the diff, test results and EXACT baseline fidelity.
- [ ] I checked final status, usable ticks, missing data and exclusion reasons.
- [ ] I did not equate changed choices with improvement, predicted cost with spend, or expected loss with a per-draw guarantee.
- [ ] I retained input, reports and evidence; claims about real outcomes require appropriate real evidence.

0 of 8 items checked. Selections are not saved when you close the page.

> **A useful result note looks like this:**
>
> “On the specified baseline commit and N valid samples, the candidate with this diff passed the targeted tests. Baseline fidelity was EXACT. Metrics X and Y changed by these amounts; Z was unavailable or adverse. This is replay evidence. Acceptance still needs code review and appropriate live evaluation.”

12 Quick reference and FAQ

Send control commands as separate messages.

Command	Purpose
`/study`	Help for declarative JSON studies; attach a file to run.
`/start` **or** `/help`	Show bot usage to an authorized account
`/check`	Check prerequisites and service access
`/guide`	Download this English playbook as HTML and PDF attachments in the same chat

Command	Purpose
`/upload`	Instructions for code/parameter files and JSON studies
`/templates`	Download eight examples: five code/parameter files and three study JSON files
`/validate`	Use as a document caption to validate without running
`/bench [options] <description>`	Generate and test a code candidate; options precede the description
`/param <ID> <values...> [options]`	Compare parameter values with the recorded control
`/status`	Current execution stage and queue
`/runs`	The five most recent history records
`/cancel`	Request cancellation of the current job after its current step. Stopping may not be immediate.
`/invite`	Owner only, in private chat: a one-use member invitation valid for 24 hours

See recent runs	Cancel the current job after its current step
`/runs`	`/cancel`

▼ Does any Word document or long description become a benchmark?

No. Use valid code/parameter sections or a versioned study JSON request. Unregistered algorithms require implementation.

▼ Does a study run the production brain?

A study runs reference algorithms on explicit inputs. Its report identifies mode=REFERENCE_STUDY and evidence type. Live outcome claims need suitable evidence.

▼ How do I repeat a study?

Reuse the same request, data, seeds and implementation. A server dataset can change; retain the frozen resolved-request.json from evidence.zip.

▼ Does /validate guarantee study success?

It checks structure, limits and supplied trace shape. Server dataset availability and evidence completeness are checked at execution. PASS can later lead to NEEDS_DATA or PARTIAL.

▼ Does /validate compile code or run Go tests?

No. The /validate attachment caption checks file structure, options, parameter IDs and value domains, without queuing an experiment. Send the file with an empty caption to execute it. Go tests and replay preflight apply to code/parameter workflows. Studies use their own schema and evidence checks.

▼ Do bench and params in one file apply the values to the new code?

Each section creates independent experiments with their own controls and reports, executed in file order. Parameter values are compared using the baseline replay binary. Testing a fixed candidate with parameter overrides requires a separate engineering workflow.

▼ Why does the activity bar not show a percentage?

Model, test, build and replay durations are not known in advance. The card shows the actual stage and elapsed time with an activity indicator, refreshing every eight seconds while running. It records the terminal state when work finishes, fails, is cancelled or is interrupted by a restart.

▼ Can I say “run the previous candidate again with 500 ticks”?

Not through the current interface. Each /bench request is independent and the model regenerates a patch. Identical descriptions do not guarantee identical diffs. Compare the diff, baseline and data between runs. There is no /rerun command.

▼ Can I describe the change entirely in Persian?

You can write a Persian description, but include exact file paths, function names and English code identifiers. Context retrieval currently relies on ASCII identifiers. A fully Persian description may retrieve too little useful context. The technical English examples in this guide are a more reliable way to specify the request.

▼ Will the bot update old tests when a formula changes?

No. _test.go files are outside candidate scope. Establish whether the new code is wrong or the intended product contract is changing. A contract and test change needs separate review. The bot reports the failure log and stops before benchmarking.

▼ Do passing tests approve the entire brain and release?

No. These targeted tests do not cover the full CI suite, every package, or live tests. Review the diff, evidence quality, relevant metrics, and live effects in proportion to the change.

▼ Zero changed decisions: does that mean the formula has no effect?

It only means the selected action set did not differ on this experiment's valid ticks. The changed path may not have been active, another gate may have determined the result, rankings may have stayed the same, or the sample may be uninformative. Inspect the diff and experiment context first.

▼ How do I open the HTML report?

Download the report from Telegram and open it in a browser. Use the CSV in Excel or a data tool; read the diff in a text editor. This guide is also a self-contained file and can be read offline.

▼ Will repeated experiments with 1.2 always give the same result?

The data, baseline identity and selected sample can change between runs, especially after syncing. With /bench, the patch is also regenerated. Compare runs using their job IDs, commits, diffs and cohorts.

▼ What happens after I find a promising candidate?

The bot does not publish the change. Take the diff and evidence through code review, CI, and live evaluation appropriate to the change's risk. This stage gives you concrete evidence for that decision. Replay alone does not authorize applying the candidate to production.

Source material: `benchmark/bot/service.py`, `planner.py`, `candidate.py`, `replay_build.py`, `code_experiment.py`, `report.py`, `file_requests.py` and `progress.py`, `study.py`, `study_math.py` and `study_schema.py`; the parameter registry and evidence in `benchmark/bot/VALIDATION.md`.

Prepared on **2026-10-31**. Check /help and actual /check results for later service changes. This file contains no passwords, tokens or private invitation links.